Table of Contents

Mini Project Report · Boyer–Moore Pattern Search and Algorithm Lab

SOURCE-BASED TECHNICAL REPORT · 20 OCTOBER 2026

SECTION	CONTENTS	PAGE(S)
2	Abstract	3
3	Introduction	4–7
4	Literature Survey	8–10
5	Proposed Methodology	11–16
6	Implementation Description	17–24
7	References	25
8	Appendix	26–31

Sections covered

- Introduction — domain, problem, objectives, scope and requirements.
- Literature Survey — classic exact matching, Boyer–Moore rules, Galil's rule and implementation trade-offs.
- Proposed Methodology — high-level view, architecture, operational flow, preprocessing, benchmark design and measurement controls.
- Implementation Description — technology stack, algorithms, web interface, command line, tests, screenshots, results and interpretation.
- References — primary publications and the official Python documentation used to prepare this report.
- Appendix — setup steps, algorithms, benchmark datasets, a worked example and verification guide.

Document note. This is an editable soft-copy report. No cover, institute name, student name, guide name, certificate or approval details have been invented.

Contents continued

FIGURES · TABLES · NOTATION

List of figures

FIGURE	TITLE	PAGE
1	BM Search system architecture	12
2	Pattern search operational flow	13
3	Right-to-left mismatch and safe-shift illustration	14
4	Real BM Search output for a 5-match example	21
5	Browser benchmark comparison chart	22
6	DNA scenario results from the application	23

List of tables

TABLE	CONTENTS	PAGE
1	Functional and quality requirements	6
2	Search strategies and heuristic combinations	9
3	Benchmark experiment variables and controls	16
4	Implementation components and responsibilities	17
5	Automated test coverage	20
6	Eight-algorithm aggregate benchmark results	24
7	Scenario-level winners	28

Notation

SYMBOL	MEANING
n	Number of characters in a text.
m	Number of characters in the pattern.
s	Start offset of the current pattern alignment in the text.
j	Pattern index at which right-to-left comparison stops.
σ	Alphabet size: number of distinct characters the input may contain.
O(·)	Asymptotic upper bound; a cost-model statement, not a timing promise.

How page numbers were prepared. The contents pages are intentionally at the front of the report; the page ranges correspond to the planned section starts. The delivered Word document is checked for its rendered page count.

2. Abstract

PROJECT SUMMARY

This mini project implements an exact literal-string search application around Boyer–Moore pattern matching. It combines a pure-Python search engine, a command-line interface and a locally hosted browser interface. The purpose is twofold: provide a usable way to locate a phrase in text and expose the algorithmic choices behind the search so their practical behaviour can be studied.

The implementation includes eight strategies: a left-to-right naive baseline, Horspool, bad-character-only Boyer–Moore, an extended bad-character variant, good-suffix-only Boyer–Moore, full bad-character plus good-suffix Boyer–Moore, full extended Boyer–Moore, and the full extended version with Galil's rule. Pattern preprocessing creates a rightmost-occurrence table or per-character sorted position lists and a strong good-suffix shift table. The search compares from right to left and advances only by a safe shift. The Galil-enabled version reuses a known matching prefix after overlaps.

A browser search accepts a literal pattern, pasted or loaded local UTF-8 text, and a selected algorithm. Results include highlighted occurrences, one-based line and column, comparison count, and measured elapsed time. The separate algorithm lab calls the actual implementations against generated binary, DNA, lowercase, ASCII-128, English-like, repetitive-DNA and adversarial inputs. Correctness is checked before performance results are presented.

For the reproduced interface experiment included in this report, six generated-text profiles contributed three pattern lengths each and four edge-case profiles contributed one pattern each. Each case contains measurements for all eight algorithms: 22 cases × 8 = 176 result rows. On this run the lowest mean runtime was measured for BM-full (2.445 ms), while BM-galil used the fewest average comparisons (0.346 per text character). The distinction demonstrates why the two metrics are reported separately. Results are local measurements, not a hardware-independent ranking.

Keywords. Exact string matching; Boyer–Moore; bad-character heuristic; good-suffix heuristic; Galil's rule; preprocessing; comparison count; empirical benchmark.

3. Introduction

3.1 · BACKGROUND AND PROJECT CONTEXT

Searching for a phrase inside a body of text is a fundamental task in source-code navigation, document search, information retrieval, data cleaning and bioinformatics. A direct approach tests the pattern at every possible starting position and scans it from left to right. This is easy to understand and gives a reliable reference result, but it may revisit the same text characters many times. When text and pattern are long, the repeated work can become significant.

A string matcher takes a text T of length n and a pattern P of length m and reports all valid start offsets i for which T[i:i+m] equals P. This project uses exact, case-sensitive, character-for-character matching. It does not assign a similarity score or infer a match under spelling variation. Making that boundary explicit is important: pattern recognition can refer to many different problems, while this implementation specifically solves literal substring search.

The Boyer–Moore algorithm takes advantage of two facts. It compares a candidate alignment starting at the pattern's rightmost character, and it uses a mismatch or an already-matched suffix to skip alignments that cannot possibly succeed. Its performance depends on the pattern, alphabet and text structure. The project therefore treats the classic algorithm not merely as a function, but as a family of variants and experiments whose observable behaviour can be inspected.

The deliverable has evolved from an algorithm-and-benchmark package into a small tool a person can actually operate: type a phrase, paste or load a local text, choose a search strategy, and inspect occurrences in context. The same application contains an experiment workspace for evaluating algorithm variants. This pair of user-facing and analytical goals gives the project both practical value and an appropriate computer-science foundation.

Project identity. BM Search is a literal pattern finder and algorithm lab; it is not an image recognizer, fuzzy-search engine, database search service, or semantic understanding system.

3. Introduction — problem statement

3.2 · MOTIVATION

A useful string-matching demonstration must satisfy two requirements that are often kept separate. First, the search results must be trustworthy and helpful to a person: every occurrence should be found, overlap must not be lost, and the user should be able to locate the match again. Second, comparisons between algorithms must be interpretable. If each algorithm is measured on different text, a different pattern, or a different machine state, apparent performance differences cannot confidently be attributed to the algorithms.

Problem statement

Design and implement an accessible local application that accepts a non-empty literal pattern and text, finds every overlapping occurrence with selectable exact-matching algorithms, and reports positions and operation counts. In addition, execute each registered implementation over a reproducible collection of text and pattern characteristics, verify that result locations agree, and present both character-comparison and elapsed-time observations in a usable interface.

Why Boyer–Moore

The naive algorithm supplies an understandable baseline and makes an independent comparison point. Horspool provides a practical bad-character shift simplified to the window’s final character. Full Boyer–Moore adds strong good-suffix knowledge to mismatch processing. Extended bad-character tables and Galil’s rule show how additional pattern structure can change shifts and comparisons. Comparing these methods highlights that algorithmic work, preprocessing costs and language-level overhead are related but distinct factors.

Intended users

The tool is appropriate for learners studying pattern matching, programmers looking up a literal phrase in a modest text file, and project evaluators wishing to reproduce the included experiments. It binds the web server to the loopback interface by default. Inputs remain local to that running process; no remote storage or external service is part of the architecture.

This scope deliberately favours clarity over a claim of production enterprise search. The application demonstrates the end-to-end path from a classic string-matching algorithm to a tested interface, and it makes the measured cost and current limitations visible.

3. Introduction — objectives

3.3 · REQUIREMENTS

The project objective is to provide a working, measurable and explainable pattern-matching application. The requirements below are derived from the current source, its automated tests and its visible user interface. They distinguish implemented capabilities from future capabilities so that the report does not suggest that approximate or semantic search exists.

ID	REQUIREMENT	IMPLEMENTED BEHAVIOUR
FR-1	Find literal occurrences	Return all start offsets, including overlapping matches.
FR-2	Show useful locations	Render one-based line and column numbers and highlight each result.
FR-3	Choose algorithm	Expose all eight registered search algorithms.
FR-4	Use direct or local-file text	Paste text or load a local file in the browser; the CLI also accepts UTF-8 files.
FR-5	Report work	Show character comparisons and elapsed time for the selected search.
FR-6	Compare implementations	Benchmark all eight implementations on common patterns and scenarios.
NFR-1	Local operation	Serve the application on 127.0.0.1 by default using the standard library.
NFR-2	Correctness visibility	Reject mismatched algorithm outputs before benchmark results are returned.
NFR-3	Readable results	Use a responsive interface, metric cards, comparison charts and a scenario table.

Non-functional expectations include predictable local resource use, explicit error messages for invalid web requests, readable result locations and repeatable pseudo-random benchmark generation. The web search currently accepts text up to 2,000,000 characters, a pattern up to 10,000 characters and one request body up to 4 MiB; the interface limits a loaded text file to 2 MiB. The benchmark permits text lengths between 256 and 100,000 characters and between one and three timing repetitions.

Success criterion. Every implementation must return the brute-force-equivalent occurrence list on tested inputs; the benchmark must include all eight variants and separate count-based from time-based conclusions.

3. Introduction — scope and use cases

3.4 · BOUNDARY AND ACCEPTANCE

Included

- Exact, case-sensitive substring matching over Python Unicode strings.
- Overlapping occurrence reporting, including well-defined empty-pattern behaviour in the lower-level algorithm API.
- Naive, Horspool and six Boyer–Moore configuration variants.
- Multiline browser search with highlighted matches and source-line context.
- Reading local files through the browser’s explicit file-selection action, plus CLI file and standard-input workflows.
- Generated benchmark corpora, edge cases, a correctness gate and CSV-capable standalone benchmark commands.

Not included

- Regular expressions, wildcards, fuzzy spelling, ranking, semantic search or language-specific tokenisation.
- Recursive folder crawling, persistent index creation, cross-file project search in the browser, or cloud synchronisation.
- Image, audio, object or face recognition; the word pattern refers to character sequences in text.
- Cross-platform packaging as an installer, a public multi-user server or an independently measured hardware performance claim.

Example use cases

A learner can paste a short passage, search for a pattern and inspect overlapping matches. A developer can load a local source file and find a literal identifier; for larger or multi-file command-line work, `python -m bmsearch find PATTERN file1 file2` prints file, line and column. An evaluator can choose Algorithm Lab, hold dataset and text size constant, run all eight variants and filter a scenario table to inspect a particular alphabet or periodic case.

Acceptance is demonstrated by the automated test suite, a live browser search that finds five deliberately placed occurrences, and a live benchmark response containing 176 records covering all eight algorithms. The demonstration proves the stated feature path; it does not establish a statistical confidence interval or guarantee identical time measurements on another machine.

4. Literature Survey

4.1 · EXACT STRING MATCHING FOUNDATIONS

String matching studies whether a pattern occurs inside a longer text and how efficiently every occurrence can be found. The straightforward scan tries n−m+1 alignments and can perform up to m character comparisons at each one; for a text of length n and a pattern of length m, its worst-case character-comparison bound is O(nm). The approach is simple, useful for short inputs, and valuable as a correctness oracle. This report's `naive` implementation checks each alignment in order and counts each actual character comparison.

Boyer and Moore's 1977 paper describes searching by beginning at the pattern's right edge. A mismatch gives more information than "this alignment failed": the pattern's character that contradicted the text and any suffix already known to match constrain how far the pattern can be shifted safely. Consequently some intermediate text windows need never be checked. Their paper describes practical average behaviour as pattern length grows while also discussing the theoretical cost model [1].

This project searches for every occurrence rather than stopping at the first. The difference matters after a match: the next alignment must preserve possible overlaps. In the input `aaaa` with pattern `aa`, valid starts are 0, 1 and 2. A matcher that moves the complete pattern length after the first result would miss two answers. Unit tests explicitly enforce the overlapping-result contract for each registered algorithm.

Preprocessing and amortisation

Boyer–Moore precomputes structures from the pattern, then uses them across a text scan. The current class is designed as a precompiled matcher: construction stores its selected bad-character data, good-suffix table and pattern period, and `search` reuses these values. This is particularly appropriate when the same pattern will be searched through several texts. The standalone functional registry favours a simple text-and-pattern call; its Boyer–Moore lambdas construct a matcher for that call.

Preprocessing has its own cost and memory footprint. A useful experimental report therefore states the metric being measured and whether setup is timed. The browser's search metric includes matcher construction inside the measured operation, while the standalone benchmark times each registered function call, which also includes its setup. Counts, timing and asymptotic bounds answer different questions and should not be presented as interchangeable evidence.

4. Literature Survey — heuristic rules

4.2 · BAD CHARACTER AND GOOD SUFFIX

Bad-character rule

Suppose the pattern aligns at offset s, right-to-left comparison fails at pattern position j, and the mismatching text symbol is c. In the classic table, the rightmost occurrence of c in the entire pattern determines an alignment that may bring that occurrence under the mismatch. The project builds the table by retaining the last index for each character. If no occurrence exists, the pattern can move past the mismatching text symbol. The computed bad-character shift is clamped to at least one to guarantee forward progress.

A common issue with the simple rightmost-occurrence calculation is that its occurrence may lie to the right of mismatch position j. The extended rule instead finds the nearest occurrence of c strictly to the left of j. The project stores ascending position lists per character and uses binary search (`bisect_left`) to find the preceding index. A missing eligible position yields shift j+1. The sparse representation uses O(m) total positions rather than a table sized for every alphabet character and pattern position; lookup is O(log m) in the number of positions for c.

Strong good-suffix rule

If the suffix P[j+1:m] already matched before a mismatch at j, the good-suffix rule shifts the pattern to align another compatible occurrence of that suffix. A strong version only reuses an occurrence when the character preceding that suffix occurrence does not recreate the same mismatch. If no suitable internal occurrence exists, a matching border can be brought into alignment. The search combines the heuristic shifts by taking their maximum, because both constraints must remain safe.

The `good_suffix_table` routine implements strong good-suffix preprocessing using a border array and fills a shift array in O(m) work and space. Entry `shift[j+1]` is the shift for a mismatch at j; entry `shift[0]` is the shift after a complete match and supplies the pattern's smallest period. A hand-verified table for `GCAGAGAG` and random-agreement tests help guard the subtle indexing and border logic.

VARIANT	BAD CHARACTER	GOOD SUFFIX	EXTRA
BM-bc	Classic	—	Ablation
BM-bc-ext	Nearest left	—	Ablation
BM-gs	—	Strong	Ablation
BM-full	Classic	Strong	Combined
BM-full-ext	Nearest left	Strong	Combined
BM-galil	Nearest left	Strong	Galil after match

4. Literature Survey — periodic patterns

4.3 · GALIL'S RULE AND THE COMPARISON GAP

A pattern that overlaps heavily with itself can expose repeated comparison work. For example, searching for `a` repeated m times inside `a` repeated n times produces n−m+1 overlapping matches. The full Boyer–Moore good-suffix table can shift from one valid occurrence to the next, yet the portion known from the previous match may still be compared again. Galil's 1979 extension tracks a region known to match after the shift; the matcher can avoid rechecking that region under the rule's conditions and maintain linear worst-case matching behaviour for repeated occurrences [2].

In this implementation, a `skip` boundary is reset on a mismatch. After a full match, a period-based shift is performed and, when Galil is enabled alongside the good-suffix rule, a known prefix boundary is carried forward. The Galil option is therefore intentionally conditional on good-suffix use. A 20,000-character repeated-a test with a 40-character pattern asserts that Galil stays below three comparisons per text character and that the unextended full variant performs over ten comparisons per text character.

Comparative perspective

The reference literature establishes algorithmic principles and complexity statements; application performance depends on concrete code and input distributions. Python dictionary lookups, allocation, preprocessing, character diversity, output size and clock resolution can affect runtime. A variant that uses fewer character comparisons may still be slower if each step performs more work. Conversely, a large result set can dominate total costs that the comparison metric does not represent.

The practical gap addressed by this project is not the invention of a new theorem. It is an inspectable, executable learning tool that puts classical algorithm variants, correctness checks and differentiated datasets in one place. The browser frontend turns the comparison into something accessible without a shell, while the pure-Python core remains small enough to read and test. The report therefore evaluates implementation evidence rather than claiming novelty beyond the established literature.

Interpretation. A worst-case bound is an asymptotic guarantee under stated assumptions; one benchmark run is a measurement for one environment and one workload.

5. Proposed Methodology

5.1 · HIGH-LEVEL VIEW

The design uses a small layered architecture. Presentation collects the pattern, source text and algorithm, then renders the response. A local HTTP layer serves the static interface and accepts JSON. Service functions validate parameters and dispatch either a line-aware search or a full benchmark. The algorithm layer supplies an independent function for each variant; the preprocessing layer constructs the pattern-derived tables. The benchmark layer creates shared datasets and checks answers before returning its measurements.

For a normal user search, the browser sends one request to `/api/search`. The service validates that the pattern is non-empty and under its limit, that the text is a string within its limit, and that the selected algorithm exists in the registry. Multiline text is divided into logical lines; each algorithm searches a line, and zero-based algorithm offsets are presented as one-based line/column positions. The JSON result also contains the match count, comparison count, elapsed milliseconds and an indication if more than 1,000 displayed matches were truncated.

The benchmark route is separate: `/api/benchmark` accepts a bounded text size and repetition count. It generates six text profiles and four adversarial cases; the registered eight algorithms run against identical text/pattern pairs. Each case records elapsed time, comparison count per text character and result count. A correctness disagreement raises an explicit failure rather than yielding a misleading chart.

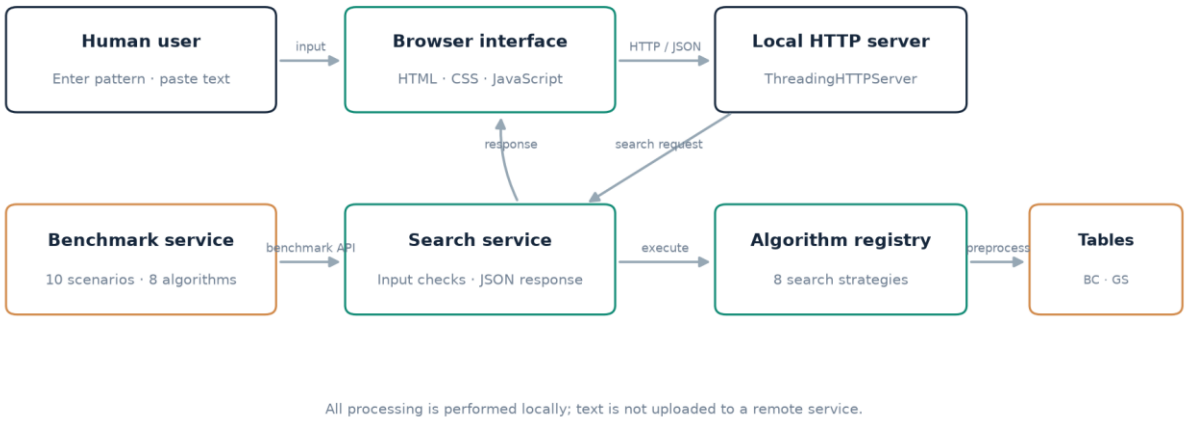


Figure 1. BM Search high-level local application architecture.

Boundary. The browser file picker reads the selected text into the page, which then submits its contents to the local server. The server itself does not crawl arbitrary user directories.

5. Proposed Methodology — detailed architecture

5.2 · LAYERS, COMPONENTS AND DATA FLOW

The browser layer consists of a static HTML document, a responsive CSS stylesheet and a small JavaScript controller. It exposes Pattern Search and Algorithm Lab, transfers request bodies as JSON, and handles API errors by showing an alert. User-supplied search-result strings are inserted through text nodes, and the matching span is highlighted with a mark element. The app does not require a frontend framework, package installation or external search service.

The HTTP layer in `web.py` derives from `BaseHTTPRequestHandler` and runs inside Python's `ThreadingHTTPServer`. It serves only four mapped paths for the page and application assets, plus a lightweight health endpoint. POST bodies have a fixed maximum size; malformed JSON and invalid input return descriptive errors. Binding defaults to 127.0.0.1 so the app is accessible on the same machine by default. The loopback default is a deployment boundary, not user authentication.

LAYER	RESPONSIBILITY	MAIN MODULES
Presentation	Form state, search results, charts and filters	web/index.html · app.css · app.js
HTTP interface	Static asset allow-list; health/search/benchmark routes	web.py
Search service	Validation, pattern dispatch and line coordinates	web.py · cli.py
Algorithms	Result offsets and character-comparison counts	algorithms.py
Preprocessing	Occurrence data, strong suffix shifts and pattern period	tables.py
Evaluation	Shared text generators, correctness gate and timing	bench.py · datasets.py
Verification	Deterministic, randomized and boundary-case tests	tests/test_algorithms.py

The returned search record is deliberately modest: an array of displayed matches contains line, column, source line and highlight offsets. Additional top-level values summarise total matches, comparisons, elapsed time and display truncation. Benchmark JSON contains scenario records and algorithm summaries. This separation gives the interface enough information to display useful observations without transferring algorithm control into browser code.

5. Proposed Methodology — operational diagram

5.3 · NORMAL SEARCH REQUEST

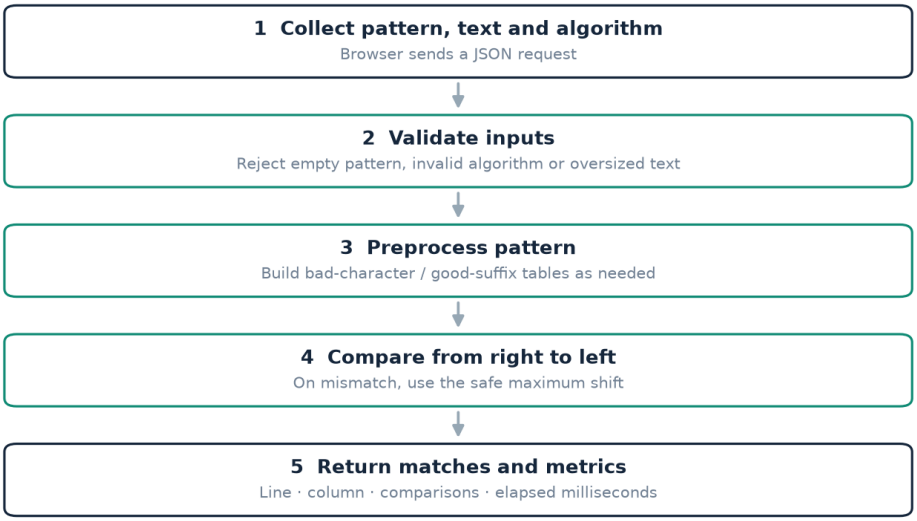


Figure 2. The operational path from user input to rendered match locations.

Sequential operating procedure

- Open the locally served page and select Pattern Search.
- Enter a non-empty literal pattern; paste text or load a supported local text file.
- Choose one registered algorithm; Boyer–Moore with Galil's rule is the initial default.
- Submit the form; JavaScript sends pattern, text and algorithm to the local search API.
- Validate limits and the registry selection; process each line using the selected matcher.
- Render a line and column for every displayed hit and summarise matches, comparisons and timing.

Line-oriented processing makes the reported column the pattern's one-based position within its source line, consistent with the command-line interface. It also means a pattern is not matched across a newline boundary. Search is case-sensitive and character based. In a Python Unicode string, positions are code-point indices, not UTF-8 byte offsets or grapheme-cluster positions; a displayed column is therefore an algorithm offset plus one.

If the search input is invalid, the API sends an HTTP 400 JSON error and the interface displays it instead of presenting a success-shaped fallback. If no occurrence exists, zero is shown and the match panel gives a no-results message. An application request may return at most the first 1,000 detailed result objects, while its overall match count is preserved and the result is labelled as truncated.

5. Proposed Methodology — matching procedure

5.4 · ALIGN, COMPARE AND SHIFT

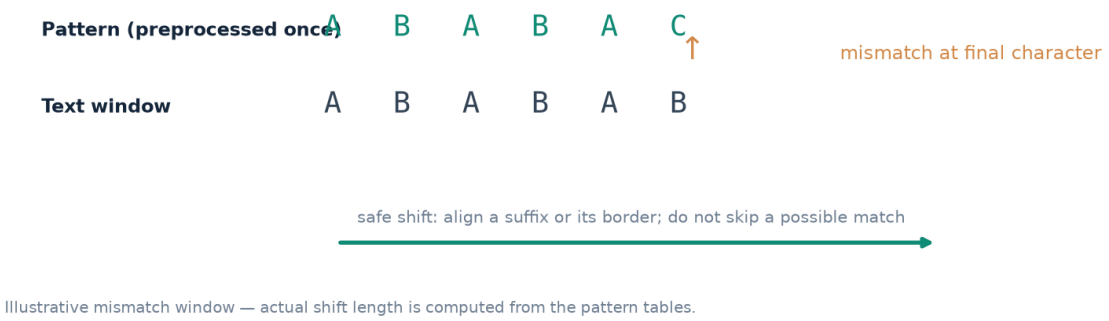


Figure 3. A right-to-left comparison discovers a mismatch after matching a suffix.

Algorithm outline

```
preprocess the non-empty pattern as required
shift = 0
while shift <= text_length - pattern_length:
    compare pattern and text from right to left
    if every position matches:
        record shift
        advance by the period shift when available
    else:
        bc = safe bad-character shift, or 1 when disabled
        gs = safe good-suffix shift, or 1 when disabled
        shift += max(bc, gs)
return all recorded offsets and the comparison count
```

For a mismatch at pattern index j, the full matcher reads the precomputed strong suffix shift for j+1. The bad-character branch either queries a rightmost-index dictionary or looks up the nearest eligible index in the character-position list. When that rule is disabled it contributes one. Taking the larger shift preserves both heuristics' safety constraints and guarantees progress. After a full match, the period entry from the good-suffix table finds the next potentially overlapping occurrence.

The implementation defines an empty pattern at the lower level as matching at every boundary from zero through n, with zero character comparisons. The user-facing API rejects an empty pattern because a GUI prompt for an empty value is more likely to be an input mistake than a request for n+1 boundary matches. This is an explicit difference between core semantics and interface validation.

All-occurrences guarantee. When a match is recorded, the period/border shift preserves overlapping matches. It is not safe to advance by the entire pattern length in every input.

5. Proposed Methodology — benchmark plan

5.5 · SHARED INPUT AND MEASUREMENT

The experiment is a paired comparison: each algorithm receives the same generated text and the same pattern in a scenario. For the six ordinary profiles, the app extracts patterns of lengths 4, 16 and 64 from positions inside the generated text, ensuring that these patterns do occur at least once. The four adversarial inputs use an eight-character pattern against a 5,000-character text at the 20,000-character setting. Thus the experiment creates 18 ordinary cases and four adversarial cases.

A deterministic random generator is seeded with 42 at the beginning of each run. Each dataset is generated in a fixed iteration order, and patterns are sampled after generation. Each algorithm measures the same case twice when the repetitions setting is 2; the existing runner keeps the best (minimum) observed wall-clock sample for the case and stores that sample in the scenario row. It also records one deterministic comparison count per algorithm/pattern. Across 22 cases, each containing eight results, a complete response has 176 records.

Before displaying the benchmark, `run_case` checks all returned match lists against a reference algorithm's list. The service also rechecks its variants when constructing the record set and treats disagreement as an error. The UI exposes a scenario dropdown, aggregate runtime and comparison charts, and a per-scenario/per-algorithm table. A result row includes text length, pattern length, hits, comparisons per character and elapsed milliseconds.

Two complementary metrics

- Character comparisons per character = the algorithm's character-comparison counter divided by text length.
- Elapsed milliseconds = observed wall-clock time of a whole function call, which includes that function's per-call preprocessing where applicable.
- Aggregate averages are calculated across scenario records; edge-case scenarios contribute one pattern each, ordinary scenarios contribute three.

Measurement rule. The repeated timings are not averaged inside each case: the current benchmark retains the fastest of the selected repetitions, and the overview then averages those per-case best values.

5. Proposed Methodology — experiment design

5.6 · INPUTS, CONTROLS AND THREATS

DIMENSION	LEVELS IN THE APP RUN	WHY IT MATTERS
Text structure	Binary, DNA, lowercase, ASCII-128, English-like, repetitive DNA	Changes alphabet size, repetition and suffix recurrence.
Pattern length	4, 16, 64 characters	Shows how pattern preprocessing and larger shifts behave.
Adversarial form	All-match periodic; missing-prefix; no-alphabet-overlap; internal mismatch	Stress overlap, mismatches and safe-shift rules.
Algorithm	Eight registered strategies	Separates heuristics and combined variants.
Text length	20,000 ordinary; 5,000 edge-case at 20,000 selected	Normalises ordinary workloads; edge-case factory receives one quarter of size.
Timing repetitions	2 timed samples per ordinary case	Provides a best-of-two value, not a robust distribution.

The main controls are common case construction, fixed PRNG seed, exact result agreement, a single application process and identical measurement code. Nonetheless, these timings remain vulnerable to CPU scheduling, process load, thermal state, interpreter version, wall-clock measurement resolution and run-order effects. The best-of-two rule reduces the influence of a single abnormally slow sample but creates an optimistic estimator and cannot substitute for medians, interquartile ranges, confidence intervals or a repeated independent-run study.

The comparison counter counts calls where a pattern character is tested against a text character. It does not count preprocessing table operations, Python bytecode, output object creation, string splitting or renderer work. A short pattern with many hits also allocates and displays more locations. The benchmark has deterministic match counts per case, but a large all-match result increases data volume and can influence both runtime and the user interface.

The run reported below uses Python 3.14.7 in this Windows workspace; the host’s CPU model, memory and power profile were not captured. Means and pairwise rankings are descriptive evidence for this run only. For more defensible comparative research, capture full per-repetition raw times and hardware metadata, randomise algorithm order, add warm-ups, report median and spread, and execute multiple independently seeded corpus runs.

6. Implementation Description

6.1 · TECHNOLOGY STACK AND PROJECT STRUCTURE

The implementation is a pure-Python package named `bmsearch` with a no-dependency run path. The declared minimum Python version is 3.9. Runtime algorithms use the Python standard library for data structures, argument parsing, file handling, JSON, timing and the threaded HTTP server. The responsive frontend uses plain HTML, CSS and JavaScript and is served as packaged static resources. `pytest` is declared as an optional development extra, while the maintained test suite is also executable with the standard-library `unittest` runner.

FILE	RESPONSIBILITY
bmsearch/algorithms.py	SearchResult, naive/Horspool/Boyer–Moore, registry of eight variants.
bmsearch/tables.py	Last-occurrence, per-character positions, extended shift and strong suffix table.
bmsearch/datasets.py	Deterministic-friendly text generators, pattern sampling and adversarial cases.
bmsearch/bench.py	Repeated timing runner, correctness agreement, console summary and CSV export.
bmsearch/cli.py	`find` command for stdin, text and UTF-8 files.
bmsearch/web.py	Local HTTP server; search and benchmark JSON API routes.
bmsearch/web/	Search form, algorithm selector, benchmark controls, chart, scenario table and responsive styling.
tests/test_algorithms.py	Algorithmic, CLI and browser-service unit tests.

Packaging metadata in `pyproject.toml` registers both `bmsearch` and `bmsearch-ui` command entry points and includes HTML/CSS/JavaScript in package data. Consequently the web assets are intended to be available both from the source checkout and after an editable or ordinary package installation. The user can run the app without a build step or JavaScript dependency installation.

Environment recorded for this report. Windows host; Python 3.14.7; local web app on 127.0.0.1:8765 during capture; text benchmark size 20,000; 2 timings; no CPU model recorded.

6. Implementation Description — algorithms

6.2 · PREPROCESSING AND MATCH ENGINE

`SearchResult` standardises the output of every algorithm as a list of zero-based match offsets plus an integer comparison count. The naive search checks candidate starts in increasing order. Horspool compares right-to-left and shifts using the character at the end of the current text window. `BoyerMoore` stores its pattern and builds only the tables required by the chosen flags. During a search, the matcher traverses right-to-left, records a full match, or calculates a mismatch shift.

The standard bad-character table maps each character to its last pattern index. The extended structure maps each character to a sorted list of its indices; `bisect_left` returns the closest strictly preceding occurrence. Strong good-suffix preprocessing stores an O(m)-length shift array derived from border information. The matcher chooses the maximum safe shift, not the minimum, since both heuristics independently describe displacements that do not skip a valid occurrence.

CONFIGURATION	BAD-CHARACTER SETUP	GOOD-SUFFIX SETUP	GALIL
naive	None	None	No
horspool	End-window shifts	None	No
BM-bc	Rightmost position	Disabled	No
BM-bc-ext	Per-character positions	Disabled	No
BM-gs	Disabled	Strong O(m) table	No
BM-full	Rightmost position	Strong O(m) table	No
BM-full-ext	Per-character positions	Strong O(m) table	No
BM-galil	Per-character positions	Strong O(m) table	After match

The full Galil variant is especially relevant when the pattern is periodic and matches overlap. It retains a known prefix length after a complete match and reduces comparisons of that already-proven region. Its benefit is workload specific: the app’s aggregate run shows fewer comparisons than the other variants, but its elapsed time is still influenced by the implementation’s additional Python-level operations.

The suite includes an empty-pattern semantic at the core level, no-match and full-match boundaries, a single-character pattern, Unicode input and overlapping matches. The API adds stricter user-input checks without changing the low-level algorithm contract.

6. Implementation Description — interfaces

6.3 · WEB, CLI AND JSON SERVICE

`python -m bmsearch.web` starts a local HTTP server and prints its address; Ctrl+C shuts it down. The default host is 127.0.0.1 and the default port is 8000. In this project session the server was run on 8765, which is why the captured screenshots display a local port that differs from the documented default. The `--host` and `--port` parameters permit explicit local configuration.

ROUTE / COMMAND	INPUT	OUTPUT / EFFECT
GET /	None	Search and benchmark user interface.
GET /api/health	None	JSON `{"status":"ok"}` health check.
POST /api/search	Pattern, text, algorithm	Match locations, totals and measurement metrics.
POST /api/benchmark	Size and repetition count	All scenario records, algorithm aggregates and winners.
python -m bmsearch find	Pattern + files, --text or stdin	File/line/column lines and total matches.
bmsearch-ui	Optional --host / --port	Installed entry point for the local browser application.

Search API validation limits the pattern to 10,000 characters, the input text to 2,000,000 characters, and the HTTP request body to 4 MiB. The benchmark limit is 256–100,000 characters and one to three timing samples. File loading in the browser is client-side and capped at 2 MiB. These constraints make costs explicit; they are not an assertion that any arbitrary pattern length will perform well.

The HTTP handler returns bad-request responses for malformed JSON or invalid search and benchmark parameters, a too-large response when the request exceeds its byte limit, and a not-found response for unmapped assets/routes. The browser handles non-success responses by rendering the error message. Normal outputs and errors remain visibly distinct.

6. Implementation Description — verification

6.4 · TESTING STRATEGY AND EVIDENCE

The project uses the standard-library `unittest` framework. The test class drives all eight registry functions and the public command-line entry point as well as the web service functions. The random equivalence test generates 4,000 small text/pattern cases from binary, three-symbol and DNA alphabets; for each one, every algorithm's match list must equal a brute-force expected list. This cross-check supplies stronger evidence than a handful of hand-picked patterns.

TEST AREA	VERIFICATION
Randomised agreement	4,000 seeded cases × all eight variants, compared with starts where text.startswith(pattern, i).
Boundaries	Empty/short text, pattern longer than text, complete match, single-character pattern and overlaps.
Unicode	Non-ASCII literal match offsets.
Good-suffix preprocessing	Known shift table for pattern GCAGAGAG.
Galil comparison bound	Repeated-a text checks Galil comparisons < 3n and non-Galil full > 10n.
Comparison usefulness	A 50,000-character large-alphabet case checks BM comparisons < n/8.
CLI	Direct text positions, multiple matching file lines and missing-file error path.
Web API functions	Multiline result coordinates, invalid pattern/algorithm, benchmark dimensions and limits.

The test suite was run with `python -m unittest discover -s tests` and completed with 14 passing tests in the project's Python 3.14.7 environment. JavaScript syntax was separately checked with `node --check bmsearch\web\app.js`, and Python source was checked using `compileall`. A real Chrome session drove the local website and validated a five-match search, the benchmark endpoint, all eight visible chart rows and 176 scenario records.

Evidence boundary. These tests validate the implemented exact-search paths and selected limits; they are not exhaustive security, concurrency, accessibility or cross-platform installation tests.

6. Implementation Description — search results

6.5 · BROWSER EVIDENCE

The first screenshot records a live browser search using pattern `ABABAC` against five short lines. The text deliberately contains a match at the beginning of lines, embedded within other characters, and two back-to-back overlapping-safe occurrences on one line; a lowercase distractor tests case-sensitive semantics. The screenshot reports five matches, 50 counted character comparisons for the selected BM-galil path, and a sub-millisecond elapsed value on this capture.

Line and column positions are one-based in the interface. The first occurrence is at line 1, column 16; the next is at line 2, column 1; the embedded occurrence begins at line 3, column 2; the two occurrences on line 4 begin at columns 1 and 7. Results preserve source text and visually mark the exact matched substring. This illustrates why line-oriented output can be more actionable than a bare list of zero-based offsets.

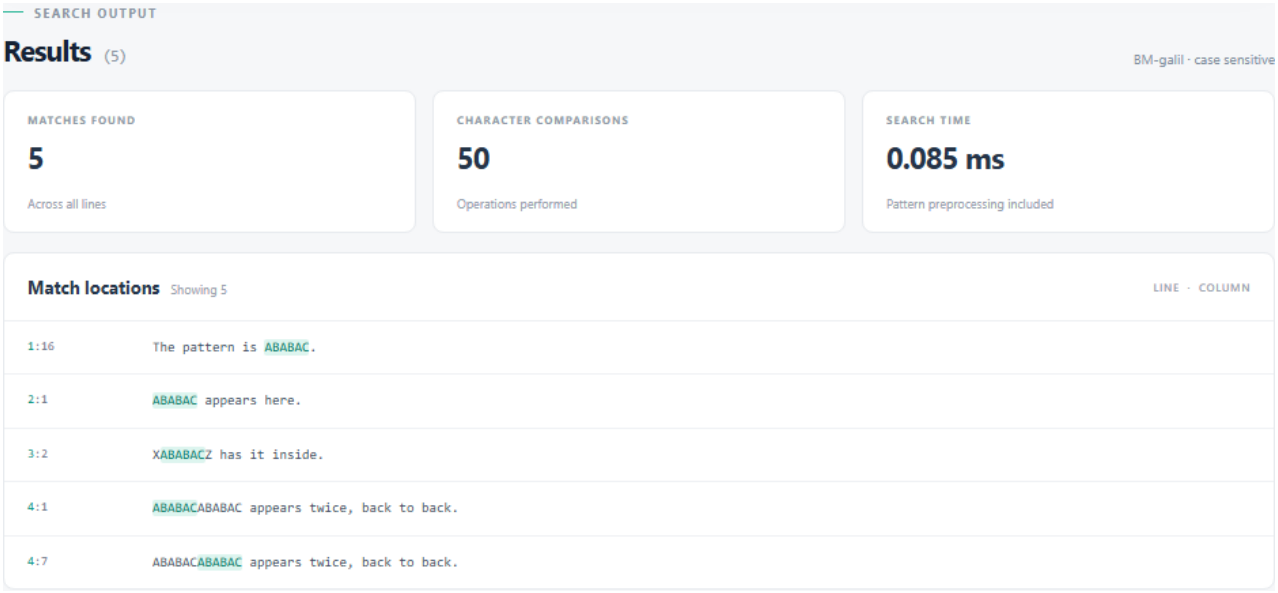


Figure 4. Search output captured from the running application; exact `ABABAC` results are highlighted.

Result interpretation. The measured comparison count is implementation-specific. All other algorithms should locate the same five start positions; they are not required to execute the same number of comparisons or take identical time.

6. Implementation Description — benchmark results

6.6 · ALL-ALGORITHM COMPARISON

The browser's algorithm overview was captured from the same application run used for the aggregate results in this report. Runtime bars are inverted so that the fastest average is the longest bar; the chart labels retain the measured millisecond values. Toggle controls in the actual interface allow the viewer to change the plotted metric from runtime to comparisons per character. The summary cards distinguish those two winners.

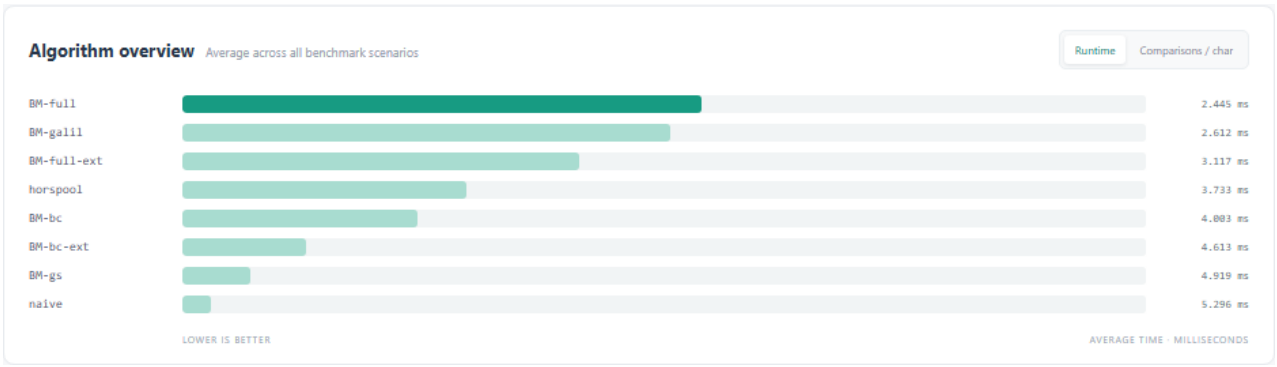


Figure 5. Runtime view from the BM Search Algorithm Lab, 20,000-character setting and two timings.

The chart is an interface screenshot rather than an idealised hand-redrawn chart. It shows all eight strategies and the values returned to the user. In this run BM-full leads average observed time, while BM-galil leads comparison efficiency. Treat the screenshot as evidence of a working, interactive report view; the numeric summary and experiment conditions on the next page are the appropriate places to make carefully qualified conclusions.

Ranking caution. At these millisecond scales rankings may change across runs or machines. No hardware-normalised performance claim is made.

6. Implementation Description — scenario table

6.7 · INSPECTING AN INDIVIDUAL TEXT PROFILE

Selecting a scenario narrows the benchmark table without changing the underlying run. The captured DNA view groups the same DNA-generated text with pattern lengths of 4, 16 and 64 and presents one row for every algorithm at each length. Matching hit counts provide an additional consistency signal for a given text/pattern pair; they do not prove that offsets are identical, so the benchmark's explicit full-list comparison is still necessary.

Scenario breakdown Per-algorithm performance by text profile

SHOW DNA

SCENARIO	ALGORITHM	TEXT / PATTERN	HITS	COMPARISONS / CHAR	TIME
DNA text	naive	20,000 / 4	74	1.336	5.740 ms
DNA text	horspool	20,000 / 4	74	0.470	3.055 ms
DNA text	BM-bc	20,000 / 4	74	0.585	4.106 ms
DNA text	BM-bc-ext	20,000 / 4	74	0.548	4.322 ms
DNA text	BM-gs	20,000 / 4	74	0.772	7.458 ms
DNA text	BM-full	20,000 / 4	74	0.470	5.227 ms
DNA text	BM-full-ext	20,000 / 4	74	0.470	5.262 ms
DNA text	BM-galil	20,000 / 4	74	0.470	5.772 ms
DNA text	naive	20,000 / 16	1	1.333	3.739 ms
DNA text	horspool	20,000 / 16	1	0.400	2.244 ms
DNA text	BM-bc	20,000 / 16	1	0.588	3.635 ms
DNA text	BM-bc-ext	20,000 / 16	1	0.502	5.218 ms
DNA text	BM-gs	20,000 / 16	1	0.440	4.862 ms
DNA text	BM-full	20,000 / 16	1	0.307	1.899 ms
DNA text	BM-full-ext	20,000 / 16	1	0.307	2.706 ms
DNA text	BM-galil	20,000 / 16	1	0.307	2.306 ms
DNA text	naive	20,000 / 64	1	1.330	5.020 ms
DNA text	horspool	20,000 / 64	1	0.340	2.560 ms
DNA text	BM-bc	20,000 / 64	1	0.383	3.798 ms
DNA text	BM-bc-ext	20,000 / 64	1	0.365	2.969 ms
DNA text	BM-gs	20,000 / 64	1	0.368	1.928 ms
DNA text	BM-full	20,000 / 64	1	0.209	1.492 ms
DNA text	BM-full-ext	20,000 / 64	1	0.209	2.002 ms
DNA text	BM-galil	20,000 / 64	1	0.209	1.871 ms

Figure 6. DNA scenario filtered in the application; each pattern-length case includes all eight algorithms.

For the 64-character DNA pattern in this case, both BM-full and BM-full-ext use approximately 0.209 comparisons per text character, while the naive strategy performs roughly 1.330. Yet each per-scenario time is a very small and noisy quantity, and some individual variants can outperform an aggregate expectation on an individual pattern. A larger pattern creates more potential work for a linear scan, while a text alphabet and its chance of mismatch affect the shifts used by Boyer–Moore and Horspool.

Reading the row. "Text / pattern" is the generated text length followed by pattern length; the time is the runner's best observed repetition for that individual algorithm/case.

6. Implementation Description — analysis

6.8 · AGGREGATE RESULTS AND INTERPRETATION

ALGORITHM	MEAN TIME (ms)	MEAN COMPARISONS / CHAR	OBSERVATION
BM-full	2.445	0.663	Fastest
BM-galil	2.612	0.346	Fewest comparisons
BM-full-ext	3.117	0.663	
horspool	3.733	1.284	
BM-bc	4.003	1.269	
BM-bc-ext	4.613	1.239	
BM-gs	4.919	0.959	
naive	5.296	1.779	

The run comprised 10 distinct scenario names: six generated text families with pattern lengths 4, 16 and 64, plus four edge cases with pattern length 8. The 20,000-character setting yields 20,000-character generated profiles and 5,000-character adversarial texts. Eight algorithms across 22 text–pattern cases produced 176 checked records. The random seed was 42; repetitions were set to two; ordinary patterns were drawn from the generated text.

On this run, BM-full had the lowest aggregate elapsed-time mean at 2.445 ms. BM-galil had the lowest aggregate comparison count at 0.346 comparisons per text character. The latter is substantially below the naive baseline, which averaged 1.779. The result is consistent with Galil's purpose on periodic overlap, but it does not show that Galil must win a Python runtime contest on every workload.

Comparison counting isolates one algorithmic operation, while wall-clock timing captures the full Python call, including per-call preprocessing and data-structure overhead. The two best-of-two samples are highly susceptible to process noise; means over 22 case minima can also hide a poor result in one data family. The scenario-detail view, identical match correctness and repeated-all-match regression test give important context that a single ranking alone cannot provide.

Responsible conclusion. For the measured run, combined Boyer–Moore offers strong observed practical performance; Galil is useful for comparison-efficiency under periodic patterns. Repeated, recorded experiments are needed before asserting a stable runtime winner.

7. References

BIBLIOGRAPHY AND TECHNICAL DOCUMENTATION

[1] R. S. Boyer and J. S. Moore, “A Fast String Searching Algorithm,” Communications of the ACM, vol. 20, no. 10, pp. 762–772, Oct. 1977. DOI: 10.1145/359842.359859. https://doi.org/10.1145/359842.359859

[2] Z. Galil, “On Improving the Worst Case Running Time of the Boyer–Moore String Matching Algorithm,” Communications of the ACM, vol. 22, no. 9, pp. 505–508, Sep. 1979. DOI: 10.1145/359146.359148. https://doi.org/10.1145/359146.359148

[3] D. E. Knuth, J. H. Morris, Jr., and V. R. Pratt, “Fast Pattern Matching in Strings,” SIAM Journal on Computing, vol. 6, no. 2, pp. 323–350, 1977. DOI: 10.1137/0206024.

[4] D. Gusfield, Algorithms on Strings, Trees, and Sequences: Computer Science and Computational Biology. Cambridge University Press, 1997. ISBN 978-0-521-58519-4.

[5] Python Software Foundation, “http.server — HTTP servers,” Python 3 documentation. https://docs.python.org/3/library/http.server.html (accessed 20 Oct. 2026).

[6] Python Software Foundation, “argparse — Parser for command-line options, arguments and sub-commands,” Python 3 documentation. https://docs.python.org/3/library/argparse.html (accessed 20 Oct. 2026).

[7] Python Software Foundation, “unittest — Unit testing framework,” Python 3 documentation. https://docs.python.org/3/library/unittest.html (accessed 20 Oct. 2026).

[8] Python Software Foundation, “time — Time access and conversions,” Python 3 documentation. https://docs.python.org/3/library/time.html (accessed 20 Oct. 2026).

[9] Python Software Foundation, “json — JSON encoder and decoder,” Python 3 documentation. https://docs.python.org/3/library/json.html (accessed 20 Oct. 2026).

[10] BM Search source project. `bmsearch/algorithms.py`, `tables.py`, `datasets.py`, `bench.py`, `web.py`, `web/` assets and `tests/test_algorithms.py`. Local project source inspected for this report, 20 Oct. 2026.

Source note. Algorithm descriptions are attributed to published work; application features, results and limits are attributed to the inspected source and reproducible tests.

8. Appendix A — getting started

A.1 · LOCAL RUN INSTRUCTIONS

The application targets Python 3.9 or newer and uses only the Python standard library at run time. From a PowerShell prompt, change to the root directory of the extracted project. Starting the server leaves it attached to that terminal; while it is running, open the printed local URL in a browser. Closing the browser tab does not stop the server; Ctrl+C in the terminal does.

```
cd "C:\Users\parne\Downloads\boyer-moore-lab\boyer-moore-lab"
python --version
python -m bmsearch.web
# Open http://127.0.0.1:8000
# Stop the server with Ctrl+C
```

To use another port, start `python -m bmsearch.web --port 8765` and use `http://127.0.0.1:8765`. The browser URL and listening server port must match; a stopped server produces a failed-fetch message because the page cannot reach its local JSON API. If that happens, restart the command in the project directory and reload the page. The health endpoint is `http://127.0.0.1:8000/api/health` on the default port.

Alternative package install and CLI

```
python -m pip install -e .
bmsearch-ui
python -m bmsearch find "Boyer-Moore" README.md
python -m bmsearch find "needle" --text "Find a needle."
Get-Content README.md | python -m bmsearch find "pattern"
```

The `bmsearch-ui` script is registered through project metadata. The command-line tool supports file paths, a `--text` argument, and standard input, and reports file/line/column locations. Selecting an algorithm is optional; `--algorithm BM-full`, for example, overrides the CLI's default Galil variant. The web interface requires no additional install when run directly from the checked-out project.

8. Appendix B — operating guide

B.1 · SEARCH AND BENCHMARK

Find a pattern

1. Choose Pattern Search.
2. Type a literal pattern into Pattern to find. Matching is case-sensitive.
3. Paste text or choose Load a text file (the browser-side file size limit is 2 MiB).
4. Choose one of the eight algorithm variants; use BM-galil for the default comparison.
5. Select Find matches and review the highlighted spans, one-based line/column and metrics.

Run the algorithm comparison

6. Choose Algorithm Lab.
7. Select a generated-text size from 10,000, 20,000, 50,000 or 100,000 characters.
8. Select one, two or three timings, then select Run all benchmarks.
9. Inspect the scenario count, runtime winner, comparison winner and eight-row chart.
10. Toggle the chart metric and filter the scenario table to compare individual cases.

The 100,000 size is permitted by the service but is a more expensive choice: each ordinary profile generates that many characters and all eight algorithms process 18 ordinary cases; naive and periodic variants can take materially longer on particular cases. The default 20,000 size is a reasonable demonstration setting. Browser timings depend on the machine and competing work.

The application is served as a local web page, not an installed public website. It does not save searches or retain a project-level index. A local loopback listener does not establish authentication for a service rebound to a network interface; keep the default local binding unless network exposure is intentionally configured and secured.

8. Appendix C — algorithm reference

C.1 · VARIANT NAMES AND COST CONTEXT

NAME	ROLE	IMPORTANT PROPERTY
naive	Reference scan	Worst-case O(nm); easy independent baseline.
horspool	Simplified bad-character	Compares right-to-left; shift keyed by window's final text character.
BM-bc	Ablation	Uses the classic last-occurrence table only.
BM-bc-ext	Ablation	Uses the nearest position left of the mismatch only.
BM-gs	Ablation	Uses strong good-suffix shifts only.
BM-full	Combined classical rules	Takes the larger of bad-character and good-suffix shifts.
BM-full-ext	Combined extended rule	Combines sparse indexed character positions and the suffix table.
BM-galil	Combined + overlap reuse	Adds Galil skip information after a full match.

The classic Boyer–Moore algorithm is commonly described with linear pattern preprocessing and strong shifts, and its search has well-known sublinear average behaviour for many practical inputs. Without Galil's optimisation, a periodic all-match sequence can force repeated comparisons; the project contains a regression test showing the difference in its selected periodic case. Complexity statements depend on the precise variant, assumptions and measure, so this table avoids assigning a blanket tight bound to every Python callable.

The rightmost occurrence dictionary stores O(m) distinct positions or fewer. The extended bad-character dictionary stores exactly one index entry per pattern character across its lists, also O(m) space; finding the nearest preceding instance uses a binary search in a single character's occurrence list. The good-suffix builder keeps two arrays of length m+1. Table construction has O(m) time and O(m) storage. Galil adds a small amount of integer state to the search loop.

For an empty pattern, core functions return every boundary offset, which produces n+1 matches. The web layer prevents that input. A pattern longer than the text returns no hits; a one-character pattern is supported; the core works on Python strings and the tests verify a Unicode example. The APIs count character offsets, not encoded byte offsets.

8. Appendix D — experiment detail

D.1 · REPRODUCING THE SCENARIO MATRIX

SCENARIO	GROUP	TEXT	PATTERN LENGTHS
binary	Text profile	20,000	4, 16, 64
DNA	Text profile	20,000	4, 16, 64
lowercase	Text profile	20,000	4, 16, 64
ascii-128	Text profile	20,000	4, 16, 64
english-like	Text profile	20,000	4, 16, 64
repetitive-DNA	Text profile	20,000	4, 16, 64
a^n / a^m (all match)	Edge case	5,000	8
a^n / b a^{m-1}	Edge case	5,000	8
a^n / b^m (no hit)	Edge case	5,000	8
a^n / split + mismatch	Edge case	5,000	8

At the 20,000 setting, edge-text length is calculated by the application as one quarter of the selected size. The adversarial factory creates all-match periodic data, a pattern with a missing leading symbol, a pattern absent from the text alphabet and a repetitive internal-mismatch pattern. The shared seed is 42. Sampling is from an existing random text window for the six ordinary profiles, while edge patterns are constructed explicitly.

Scenario winners on the captured run

SCENARIO	GROUP	n	FASTEST	ms	FEWEST CMP	CMP/CHAR
binary	Text profile	20,000	BM-gs	3.535	BM-gs	0.490
DNA	Text profile	20,000	horspool	2.620	BM-full	0.329
lowercase	Text profile	20,000	horspool	1.150	BM-full	0.136
ascii-128	Text profile	20,000	BM-full	1.158	horspool	0.115
english-like	Text profile	20,000	BM-bc	1.428	BM-full	0.161
repetitive-DNA	Text profile	20,000	BM-full	1.998	BM-full	0.263
a^n / a^m (all match)	Edge case	5,000	BM-galil	2.669	BM-galil	1.000
a^n / b a^(m-1)	Edge case	5,000	BM-gs	1.065	naive	0.999
a^n / b^m (best case)	Edge case	5,000	BM-gs	0.304	horspool	0.125
a^n / a^(m/2) b a^(m/2)	Edge case	5,000	BM-gs	1.352	BM-gs	0.999

Scenario winners are descriptive minima: small runtime differences may be noise, and finding the minimum from eight candidates introduces a winner’s bias. Use the aggregate chart, per-case sample values, comparison counts and repeated measurements together rather than extrapolating the fastest one-case result.

8. Appendix E — worked example

E.1 · RESULT COORDINATES AND OVERLAP

Example text

```
pattern = "ABABAC"
1  The pattern is ABABAC.
2  ABABAC appears here.
3  XABABACZ has it inside.
4  ABABACABABAC appears twice, back to back.
5  ababac is lowercase, so it will not match.
```

The expected one-based line/column positions are (1,16), (2,1), (3,2), (4,1) and (4,7). Line 1 contains 15 characters before the pattern; line 2 begins with the pattern; line 3 contains a leading X; line 4 concatenates two six-character occurrences; line 5 deliberately differs in case. This concise fixture checks literal exactness, case sensitivity, embedded occurrences and multiple hits on a single line.

Offset relationship

```
column = zero_based_match_start + 1
line_number = one_based line index
end = zero_based_match_start + pattern_length
```

The core algorithm returns a start offset within a line. The web service enumerates lines starting at one, obtains zero-based offsets for that line, and returns one-based columns. The browser uses the same offset and the selected pattern length to split each source line into prefix, highlighted substring and suffix. The CLI’s rendering similarly adds one for the displayed column. The distinction between core offsets and presentation coordinates avoids off-by-one ambiguity.

Although the example contains successive occurrences on line 4, they do not overlap: the pattern has length six and the second match begins at the next six-character boundary. The dedicated unit case `aaaa` searched for `aa` provides a true overlapping example; its expected starts are 0, 1 and 2. The all-match adversarial benchmark further evaluates a highly periodic overlap workload.

Manual check. Running the provided example in the browser should report exactly five hits. Selecting another algorithm should change performance figures, not the expected locations.

8. Appendix F — completion and future work

F.1 · DELIVERABLE NOTES

The completed mini project contains a correct, tested exact matcher family; optimized bad-character and strong good-suffix preprocessing; the Galil extension; a real local search interface; a selectable command-line workflow; and an algorithm laboratory. The test suite verifies algorithm agreement against a brute-force reference, core edge cases, preprocessing, an adversarial comparison property, CLI operation and the browser API services. This report records real screenshots of the search view, aggregate benchmark chart and scenario breakdown, and makes its run configuration explicit.

Limitations to state when presenting

- A literal pattern finder is not a general-purpose similarity or multimodal recognition system.
- Search is line-oriented in the web API; a pattern spanning a newline is not found.
- The scenario generators are synthetic and cannot represent every natural-language or production corpus.
- The fastest-of-two timing rule favours optimistic minima and the project does not record CPU model or confidence intervals.
- The browser renders up to 1,000 detailed hits even when it reports a larger true total; the CLI can search explicitly named UTF-8 files.
- A local threaded development HTTP server is not a hardened multi-user or internet-facing deployment.

Reasonable next steps

Future work can add repeated seeded trials, warm-up iterations and exported raw UI data; randomised execution order; summary medians and dispersion; target-hardware details; and a user-downloadable CSV. Product extensions could include multi-file selection, explicit whole-word/case-folding switches, cross-line matching and a cancellable worker for the largest inputs. Each extension should preserve a clear distinction between exact and approximate matching and add tests for its changed semantics.

The report is deliberately coverless: the first page is the table of contents, following the request to omit an unavailable front page. Institution, student, course, supervisor, certificate and declaration fields are omitted rather than fabricated. The file is an editable Microsoft Word document intended as a soft copy.

End of report. BM Search — Boyer–Moore String Matching & Pattern Lab · source-based project documentation · 20 October 2026.